Multi-Stack

Infrastructure Project

Voting Application on AWS

Docker | Terraform | Ansible | AWS

João Ribeiro & Irene Romero

Ironhack · Cloud & DevOps Bootcamp · 2026

What We Built & Why

A fully automated, cloud-native deployment of a multi-service voting application on AWS — using Infrastructure as Code and Configuration Management from scratch.

🐳 Containerisation

We uploaded the services to DockerHub and then pull it through ansible inside the instance that needed them

☁️ Cloud Infrastructure

Provision a secure 3-tier AWS architecture using Terraform modules.

⚙️ Configuration Mgmt

Automate container deployment across EC2 instances using Ansible playbooks

🔒 Security

Isolate services using private subnets, bastion host and least-privilege security groups

How We Did It

1 Docker

We upload custom images for Vote (Python), Result (Node.js) and Worker (.NET). And then pull it from the instances with Ansible.

2 Terraform Modules

Created reusable modules for VPC, Security Groups and EC2 instances. Remote state with S3 + DynamoDB.

3 AWS Architecture

3-tier setup: Public (ALB + Bastion), App (Vote + Result), Data (Redis + Worker + PostgreSQL).

4 Ansible Playbooks

Automated Docker installation and container deployment across all 5 instances via Bastion ProxyJump.

5 ALB Routing

Application Load Balancer routes /vote and /result paths to separate EC2 instances in private subnets.

Problems We Solved

⚠️ SSH encoding issues in ~/.ssh/config edited in VS Code caused invisible characters that broke Ansible connectivity	✓ Always recreate the file via terminal using cat heredoc to avoid non-breaking spaces
⚠️ .terraform/ folder (674MB) committed to GitHub accidentally, blocking all pushes	✓ Used git filter-branch and force push to rewrite history and remove the binary from all commits, and create a .gitignore
⚠️ Moving resources into Terraform modules while root was still referencing the originals — breaking the state and causing undeclared resource errors	✓ Rewrite outputs.tf to pull values from module outputs, giving root access to the data it needed
⚠️ Result app used PG_HOST instead of DB_HOST — environment variable name mismatch	✓ Inspected docker inspect to find the actual env vars the image expected and corrected the playbook

The Stack

Docker

Containerised each service for consistency across dev and prod environments. Multi-stage builds for .NET.

Terraform

IaC with reusable modules. VPC, SGs and EC2 defined as code — reproducible with one command.

Ansible

Agentless config management over SSH. Playbooks deploy containers to private instances via Bastion.

AWS EC2 + ALB

Path-based routing on ALB sends /vote and /result to separate private instances. Auto-healing ready.

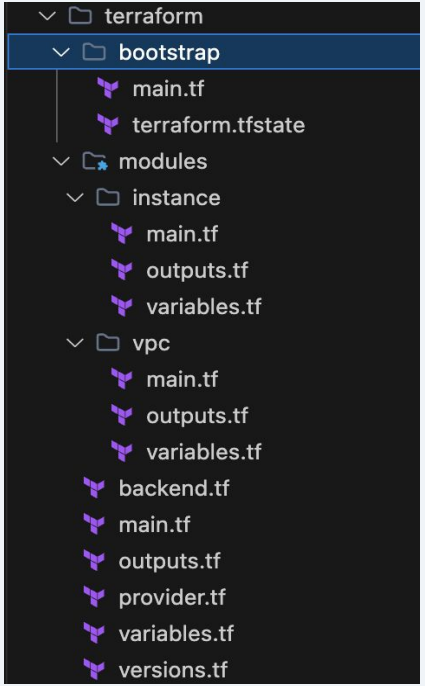
S3 + DynamoDB

Remote Terraform state with locking — safe for team collaboration and prevents state corruption.

GitHub

Version-controlled infrastructure. Full history of every change to Terraform, Ansible and Dockerfiles.

Terraform Structure



We created a terraform structure with a bootstrap that runs first of all the S3 and the DynamoDB and also set the providers.

We also created a backend.tf to tell terraform how to use that infrastructure we created.

At first, we created just a main.tf where everything worked together and after that we created a module for instances and another one for vpcs, to make it more scalable.

Ansible Structure

1. - Configuration for the playbook;
2. - Inventory: groups (bastion, backend, database, vote and result);
3. - Where is the main ansible playbook;
4. - Variables used on the playbook task

How We Worked Together

What Worked Well

→ Pair programming on complex issues — SSH, Security Groups and Terraform modules

→ Shared GitHub repository with clear commit messages tracking progress

→ Dividing work by layer: one focused on Terraform, other on Ansible and Docker

→ Using the bootcamp labs as a foundation and building on top iteratively

What Could Improve

→ Better branching strategy — working directly on main caused occasional conflicts

→ Writing a .gitignore before running terraform init to avoid large file issues

→ Documenting decisions in real-time rather than retrospectively

→ More frequent integration testing between Terraform outputs and Ansible inventory

The key routing

🚨 What Happened

Everytime we change who run the ansible playbook, we forget to change the route to the key. Sometimes it had to be /home/joao and other times /home/irene

💡 What We Learned

Always check the route

For ansible hosts it’s not the same ~ than home/user

🔄 What We'd Do Differently

We change the route everytime we change the playbook user

Key Takeaways

✅ Project met all learning objectives — containerisation, IaC, configuration management and cloud security
✅ Also some add-ons — volume for PostgreSQL, Load Balancer, S3 + DynamoDB locking

🧠 Docker networks ≠ AWS networking

Same-instance containers talk via Docker network. Cross-instance always uses AWS private IPs.

📝 IaC is only as good as the state

Remote state with S3 + DynamoDB locking is essential for any team working on the same infrastructure.

🔍 Debug at the right layer

Most issues were in the glue between tools — SSH config, env var names, port mappings.

🤖 AI accelerates but doesn't replace understanding

Using AI to help on the project still requires enough knowledge to validate and debug what it produces.

Thank You

MultiStack Infrastructure Project

João Ribeiro & Irene Romero

Ironhack · Cloud & DevOps Bootcamp · 2026

Questions?

github.com/joaodmorgadoribeiro-del/MultiStack-Infrastructure